

CS 1674 INTRODUCTION TO COMPUTER VISION ASSIGNMENT 3: AUTOSTITCH

This assignment is a coding assignment. Please read the instructions carefully and follow them step-by-step. If you have any queries regarding the understanding of any questions, please contact the TAs as soon as possible.

Due on: 23:59 PM Tuesday, April 29th.

1 Introduction

Welcome to this assignment, where you will build an image stitching system to skillfully combine a set of horizontally overlapping photos into a spectacular panoramic image. You will use the powerful ORB feature detector and descriptor to identify key features in the image and find the correspondence between photos through feature matching. Then, use the RANSAC algorithm to automatically align these photos and accurately determine their overlapping areas and relative positions. Finally, you will use image fusion technology to seamlessly stitch the aligned images together to create a stunning panoramic effect, such as the following example:

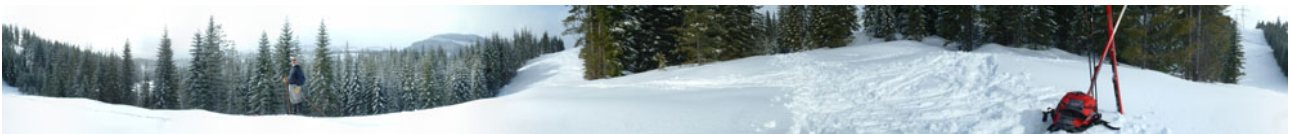


Figure 1: Panorama image example

Note that in this assignment, we will provide a set of source images to stitch for panorama. However, we strongly encourage you to shot source images yourself, which can be done by rotating a cell phone camera. In this case, your work will be more distinctive and outstanding.

Summary of potentially useful functions:

- `np.divide`
- `np.eye`
- `np.ndarray`
- `np.dot`
- `np.linalg.svd`
- `cv2.warpPerspective`

2 Details

The steps required to create a panorama are listed below:

1. Prepare source images
2. Extract features
3. Match features
4. Align neighboring pairs using RANSAC

5. Write out list of neighboring translations
6. Correct for drift
7. Read in warped images and blend them
8. Crop and save the result

More specifically, to compute the final homography here are the **13 TODO blocks** in the code snippets we provided, please DO NOT modify the code outside the TODO blocks:

- **TODO 1 (*computeHomography* in *alignment.py*):** This function takes two feature sets from image 1 and image 2, *f1* and *f2* and a list of feature *matches* to estimate a homography from image 1 to image 2. In this loop, fill the matrix *A*, which is used to construct the homogeneous linear equation system $Ah = 0$ to solve the homography matrix. You need to fill the corresponding elements of the matrix *A* according to the coordinates of the matching feature point pairs. You can access the matrix elements using indices, such as $A[0,0]$.
- **TODO 2 (*computeHomography* in *alignment.py*):** Compute the homography matrix *H* with the results of the SVD. You need to extract the appropriate elements from the results of the SVD and arrange them into a 3x3 matrix. Specifically, the homography matrix *H* is related to the eigenvector corresponding to the smallest eigenvalue of the matrix $A^T A$. In the results of the SVD. You need to reshape this vector into a 3x3 matrix.
- **TODO 3 (*alignPair* in *alignment.py*):** This function takes the two feature sets *f1* and *f2*, the a list of feature *matches*. Then it estimates and returns the inter-image transform matrix *M*. Write a complete implementation of this function. You need to implement a RANSAC loop that contains the following steps:
 1. Randomly select a minimum set of matching feature points. For homography, the size of this set is 4. Call *computeHomography* to estimate the transformation matrix using the selected matching points.
 2. Call *getInliers* to compute the inliers. After the RANSAC loop is finished, find the transformation matrix with the most inliers.
 3. Then, call *leastSquaresFit* to re-estimate the transformation matrix using all the inliers by least squares. In the end, return this transformation matrix *M*.
- **TODO 4 (*getInliers* in *alignment.py*):** For a given transformation matrix *M* and RANSAC threshold *RANSACthresh*, determine whether the i_{th} matched feature point in *f1* is close enough to its matching point in *f2* after transformation by *M*, e.g., determined by Euclidean distance less than or equal to *RANSACthresh*. If so, add index *i* to the *inlier_indices* list.
- **TODO 5 (*leastSquaresFit* in *alignment.py*):** Using all the inliers found in *inlier_indices*, compute the final homography matrix *M*. You need to first extract the matching points corresponding to *inlier_indices* from the *matches* list, and then pass these inlier matches to the *computeHomography* function.
- **TODO 6 (*imageBoundingBox* in *blend.py*):** This function takes an image *img* and a transformation matrix *M* and computes the bounding box of the image after applying the transformation. You need to compute the coordinates of the four corner points of the transformed image and find the minimum and maximum X, Y values of these coordinates to determine the bounding box's *minX*, *minY*, *maxX*, and *maxY*.
- **TODO 7 (*getAccSize* in *blend.py*):** Iterate over all the input *ImageInfo* objects. For each image and its corresponding transformation matrix *M*, you need to call the *imageBounding-Box* function to get the bounding box of the transformed image. Then, based on the bounding

box of the current image, update the global minimum X (*minX*), minimum Y (*minY*), maximum X (*maxX*), and maximum Y (*maxY*) values to find the smallest rectangular area that can contain all the transformed images.

- **TODO 8 (*accumulateBlend* in *blend.py*):** This function adds the input image *img* to the accumulator *acc* according to the given transformation matrix *M*. You need to iterate over the transformed area of *img* and add the pixel values of *img* to *acc* according to certain weights. The *blendWidth* parameter defines the hat function weight in the horizontal direction, which is used to achieve smooth transitions between images. The first three channels of the accumulator *acc* are used to record the sum of the weighted pixel color values, and the fourth channel is used to record the sum of the weights.
- **TODO 9 (*normalizeBlend* function in *blend.py*):** This function takes the accumulator image *acc* and normalizes its RGB values. Normalization requires dividing the sum of the color values of each pixel by the fourth channel of *acc* (the sum of the weights) to get the final blended image. You need to create a new image *img* to store the normalized result and divide the RGB value of each pixel in *acc* by its corresponding weight value.
- **TODO 10 (*blendImages* in *blend.py*):** In this function, you need to fill the appropriate entries in the affine transformation matrix *A* according to the value of the *is360* parameter. If *is360* is *True*, you need to implement the following operations: Translate the image to the left by a certain amount to trim off the left edge caused by accumulation. Correct the vertical drift that may exist in 360 degree panoramas. Note: *cv2.warpPerspective* performs a forward mapping, which means that the matrix *A* is an affine transformation that maps the accumulator coordinates to the final panorama coordinates. You need to calculate the drift using the *computeDrift* function.
- **TODO 11 (*parse-args* in *main.py*):** In *parse-args*, you need to customize the default values of command line arguments. In the current code, the default values of *-output*, *-blend-width*, *-ransac-rounds*, and *-ransac-thresh* are empty. You need to set appropriate default values for these parameters according to the needs of the project and the results of experiments. For example, you can set a default file name for *-output*, and set reasonable default values for *-blend-width*, *-ransac-rounds*, and *-ransac-thresh*.
- **TODO 13 (*main* in *main.py*):** after the panorama is fused, you need to customize the image cropping to crop out the black edges that appear in the generated panorama according to the situation. You can use OpenCV's image slicing function to achieve cropping. You need to determine the starting and ending coordinates of the cropping, or the number of edge pixels that need to be cropped, and use this information to intercept the valid area of the panorama.

After completing all the TODO blocks, your code should compute a panorama with good quality, from the source images provided yourself. More specifically, your code should contain:

- ***alignment.py*:** The implementation for aligning pairs of images using feature matching and homography estimation.
- ***bleng.py*:** The image blending techniques to create a seamless panorama from the aligned images.
- ***main.py*:** The main script to run the panorama stitching process, handling argument parsing, image loading, alignment, blending, and saving the final panorama.
- ***/source_images*:** A directory intended to store the input images that will be used to generate the panorama, be sure to include the source images.
- ***/results_images*:** A directory where the generated panorama images will be saved.

As usual, only *main.py* will be run during marking.

3 Submission

In this assignment, please submit all your code. Please compress your submission into a .zip file, and name the .zip file as:

A3_SECTIONNUMBER_YOURID_YOURNAME

For instance, A3_01_2023141520000_Sofia.zip

4 Marking

For this assignment, marking will be based on two parts:

- The quality of your panorama. This will be done by running your code submission on the TAs' machines, based on the script file *main.py* only.
- Your performance during the live Q&A session, where the TAs will ask random questions related to coding or theories connected to this assignment. Please note that during the live Q&A session, you should NOT rush in the queue, the order of having the session will be based on the ascending of your student numbers. Also, if you feel that the TAs are treating you with uneven standard, you can collect evidence and report to the lecturer.